SANPO USB 转 CAN

English

SANPO USB 转 CAN 协议用于在 Windows 和 Linux 上控制 CAN 设备，CAN数据转发给设备，无需驱动。发送报文在 原始CAN数据帧 外附加 SANPO 自定义帧头/尾，附加帧只作为开发板识别使用，下发给 CAN 设备的只有原始 CAN 数据，不会加入 SANPO 帧头/尾。

SANPO USB 转 CAN FD 请阅读 SANPO USB 转 FDCAN。

连接

接入开发板USB后，会出现 6 个 USB 串口，其中 2 个管理口用于本协议(Windows 下通常显示为 `SANPO Studio Management Port`)：一个访问 CAN-1/2，另一个访问 CAN-3/4；其余 4 个是 RS485 独立串口。操作系统分配的 `COM` 或 `/dev/ttyACM*` 编号不保证与板上丝印顺序一致，首次使用时建议只连接一个 CAN 设备确认对应关系。
Linux 命令模式下，也可以按照串口识别工具使用说明运行 Python 程序，识别哪 2 个串口是管理口。

示例程序：发送数据前设置 CAN 波特率

安装依赖：

```
python -m pip install pyserial
```

下面的示例代码会自动找到并通过2个管理口设置 CAN-1～CAN-4 的波特率。示例把经典 CAN 设置为 1 Mbps；第二个 5 Mbps 参数只供FDCAN使用，不影响经典 CAN 通信。

```
import re
import time
import serial
from serial.tools import list_ports

COMMANDS = ["AT+SETFDCAN=1000000,5000000"]

def is_management_port(port):
    if (port.vid, port.pid) != (0x1209, 0x2323):
        return False
    info = " ".join(str(getattr(port, name, "") or "")
                    for name in ("hwid", "interface", "location")).upper()
    return ("MI_01" in info or "MANAGEMENT PORT" in info or
            re.search(r":(?:X|\d+)\.1(?:\D|$)", info) is not None)

def send_at(stream, command):
    stream.reset_input_buffer()
    stream.write(command.encode("ascii") + b"\r\n")
    stream.flush()
    deadline, reply = time.monotonic() + 1.0, bytearray()
    while time.monotonic() < deadline:
        reply.extend(stream.read(stream.in_waiting or 1))
        if b"OK\r\n" in reply or b"ERR," in reply:
            break
    text = reply.decode("ascii", errors="replace").strip()
    if "OK" not in text:
        raise RuntimeError(f"{stream.port}: {command}: {text or 'no reply'}")
    print(f"{stream.port}: {command} -> {text}")

ports = sorted(p.device for p in list_ports.comports()
               if is_management_port(p))
if len(ports) != 2:
    raise SystemExit(f"Expected 2 SANPO management ports, found {len(ports)}:
{ports}")

for device in ports:
    with serial.Serial(device, 1_000_000, timeout=0.05, write_timeout=1) as stream:
        time.sleep(0.05)
        for command in COMMANDS:
            send_at(stream, command)

print("CAN-1..CAN-4 configured successfully.")
```

需要其他经典 CAN 波特率时，只修改 `COMMANDS` 中的第一个数字。代码中打开 管理口使用的 `1_000_000` 只是 STM32 CDC 参数，不会设置 CAN 或 RS485 的物理 波特率；物理总线参数只由明确的 `AT+SETFDCAN` 命令修改。

参数	支持值
CAN波特率	`1000000`、`500000`、`250000`、`125000`、`100000`

CAN 扩展帧

发送和接收使用相同格式：

字段	长度	说明
帧头	2 字节	固定 `45 54`，即 `ET`
Channel	1 字节	CAN 接口编号
CAN ID	4 字节	29 位扩展帧 ID，高字节在前
数据长度	1 字节	`0`～`8`
CAN 数据	0～8 字节	长度必须与前一字段一致
帧尾	2 字节	固定 `0D 0A`

向 Channel 1、扩展帧 ID `0x0000FD01` 发送 8 字节数据：

```
45 54 01 00 00 FD 01 08 01 02 03 04 05 06 07 08 0D 0A
```

CAN 标准帧

发送和接收使用相同格式：

字段	长度	说明
帧头	2 字节	固定 `53 54`，即 `ST`
Channel	1 字节	CAN 接口编号
保留字段	2 字节	固定 `00 00`
CAN ID	2 字节	11 位标准帧 ID，高字节在前
数据长度	1 字节	`0`～`8`
CAN 数据	0～8 字节	长度必须与前一字段一致
帧尾	2 字节	固定 `0D 0A`

向 Channel 2、标准帧 ID `0x142` 发送 4 字节数据：

```
53 54 02 00 00 01 42 04 11 22 33 44 0D 0A
```

小米CyberGear电机 示例

完整示例：USB 转 CAN CyberGear 使用说明

Windows 示例：

```
python usb2can_cybergear_demo_v8.py --port COM8 --motors 1 --channel 1
```

Linux 示例：

```
sudo python3 usb2can_cybergear_demo_v8.py --port /dev/ttyACM0 --motors 1 --channel 1
```

请根据实际串口、接口和电机 ID 修改参数。

常见问题

现象	处理方法
找不到 USB 串口	检查 USB 线、驱动、控制部分供电，并重新插拔开发板
发送后没有回复	检查 USB 串口对应关系、Channel、CAN ID、波特率、终端电阻、接线和设备供电
报文被发到两路 CAN	Channel 为 `0x00`；改为具体通道
回包 Channel 与板上 3/4 不一致	回包使用当前 MCU 的本地 `1/2`，属于正常行为
回包格式与本页不同	按文本发送 `AT+ET`，收到 `OK` 后重试